Making a Game: Building Your Own Game on the Engine

A companion guide to Building an AMIGA Game Engine

Richard Shannon

AmigaGameEngine v1.0

Contents

1. What You Need 1

2. Start From the Starter 2

3. The Two Entry Points 2

4. Your State: the `Variables` Block 2

5. Things Every Game Must Define 3

6. Hooks: Where the Engine Calls the Game 3

7. Adding Features: Include the Subsystem 4

Scrolling sideways 4

8. Levels and Assets 5

9. Keeping It Working 5

10. A Path From Starter to Game 5

Duck Rescue is one game built on AmigaGameEngine; this guide is for the next one. *Meadow Run* (`games/meadow_run/`, book Chapter 36) is a worked example of the whole path: a sideways-scrolling platformer that began as a copy of the starter and is laid out like Duck Rescue. It takes you from the minimal starter that boots and quits, to a game with its own state, its own subsystems and its own art, and explains what the engine expects from a game at each step.

There is no plugin system or registration table. The engine and the game are assembled together as one program (see the Single Translation Units Guide), so "the interface" is simply the set of symbols each side must define. Get one wrong and `vasm` stops with an unresolved symbol, which is a far better failure than a crash on the Amiga.

`engine/ENGINE_CONTRACT.md` is the terse reference for everything below: every hook, binding and buffer in one table. This guide is the walkthrough. Name everything you add by the rules in the Coding Conventions Guide.

1. What You Need

- **Assembler and linker:** `vasmm68k_mot` and `vlink`, on `PATH`, in the directory named by `AMIGA_TOOLCHAIN`, or bundled with the amiga-assembly VS Code extension (found automatically).
- **Python 3** with Pillow (`pip install Pillow`) for the build driver and the asset converters.
- **An emulator profile:** the WinUAE profiles in `.vscode/launch.json` (see the Launch Profiles Guide), including the AROS profile that needs no Kickstart ROM.

2. Start From the Starter

```
games/starter/
  main.asm                 entry point, INCDIR, Copper list and gradient, HELLO / WORLD, BSS
  include/game_hooks.asm       every Game*Hook the engine can call, as empty stubs
  include/core/variables.asm   the Variables block (engine fields only)
  include/hero_font_data.asm   the big font's tables (generated)
  assets/font/hero_font.bin    the big font's glyphs (generated)
  tools/make_hero_font.py      makes both from Duck Rescue's Michroma font, at 80%
```

The starter takes over the machine, paints a Copper gradient behind a 5-bitplane screen, flashes HELLO / WORLD with the engine's big-font routines (`engine/graphics/bigfont.asm`, the ones behind Duck Rescue's GAME OVER), and returns cleanly to AmigaDOS when you press ESC. It has no sprites, tilemap or sound. Copy it rather than Duck Rescue, which brings a complete game's worth of code and assets you would have to cut out again.

```
xcopy /E /I games\starter games\my_game
python tools/build.py --game my_game
```

`tools/build.py --game my_game` assembles `games/my_game/main.asm` from the repository root and links `uae/dh0/main`. Before it changes anything, edit the `incdir` lines in your `main.asm`: every `games/starter` becomes `games/my_game`.

To debug in VS Code, point the root `main.asm` (which the VS Code build task assembles) at your game: `include "games/my_game/main.asm"`.

3. The Two Entry Points

The engine calls exactly two routines by name, and supplies everything else itself:

Routine	Called	Job
`GameInitHook`	once, at the end of `EngineInit`	install your first Copper list, set up initial state
`GameStatusRun`	every frame, from `MainLoop`	your whole game

`GameStatusRun` can be three lines (the starter tests ESC and sets `QuitFlag`), or a full state machine. To use Duck Rescue's state dispatcher (27 states: loading screen, title, instructions, level transitions, gameplay and end screens), include `engine/gameplay/gamestatus.asm`. **Something must set `QuitFlag(a5)`,** or `MainLoop` never returns to `QuitToOS`.

The boot sequence around them is fixed. `Main` saves the OS registers and calls `SystemSave`. `Restart` then resets the chips, switches to `GameStack`, calls `EngineInit` (hardware detection, keyboard, fault traps, `GameInitHook`), starts the VBlank interrupt and enters `MainLoop`. You never set `a5` or `a6` yourself: `Restart` loads them once and every routine relies on them from then on.

Register	Holds
`a5`	the `Variables` block (Fast RAM)
`a6`	`CUSTOM` (`$dff000`)
`a4`	the current player struct, set per call (`lea Player(a5),a4`)
`a3`	the current actor struct, set per call

4. Your State: the `Variables` Block

All game state lives in one Fast RAM block addressed from `a5`, laid out with `rs` directives. The engine's fields and the game's fields share **one** `rsreset` chain:

```
    include    "engine_variables.i"     ; engine fields first: opens the chain
    include    "entity_variables.i"     ; only if you include entities.asm

Score:              rs.l    1           ; your fields, on the same chain
Lives:              rs.w    1
```

```
Variables_Sizeof:   rs.w    0           ; closes the block
```

Reserve the space in `main.asm`: `Variables: ds.b Variables_Sizeof` in the `mem_fast,bss` section. Never start a second `rsreset` in this file: every field after it would overlap the engine's.

Size every table from the engine's limits (`MAX_GAME_MAP_SIZE`, `MAX_ACTIVE_*`), never from one level's dimensions. A table that fits today's levels overflows silently on a taller one and overwrites whatever follows it. That is how Level 6 once corrupted the Player record.

5. Things Every Game Must Define

The starter already has all of these. Keep them when you change things:

Symbol	Where	Why
`Variables`, `GameStack`/`GameStackTop`, `Keys` (256 bytes)	`mem_fast` BSS	`Restart` and the keyboard handler use them by name
`NullSprite`	`mem_chip` BSS	unused sprite channels point here
`FaultScreen`	Chip RAM, 20,608 bytes or more	the fault screen draws here after a crash; bind it to a screen buffer
`cpGameDIW`	Copper list	`DetectNTSC` patches `DIWSTRT`/`DIWSTOP` here: two word pairs, back to back
`cpSprites`	Copper list	`ClearSprites` writes all 8 sprite pointer pairs here: 16 word pairs
`Sinus`/`SinusEnd`	any data section	`engine_const.i` computes `SINE_ANGLES` from its length
every `Game*Hook`	`game_hooks.asm`	the engine calls each by name (§6)
every `AudioPlay*` routine	your audio file	engine screens call them; an `rts` is enough

The starter keeps its own names for the last two (`StarterCopperDIW`, `StarterCopperSprites`) and binds `cpGameDIW` and `cpSprites` to them with two equates; a label with the contract name works just as well.

Every Copper list should also reset `FMODE`, `BPLCON3` and `BPLCON4` at the top (`$01fc,$0000`, `$0106,$0c00`, `$010c,$0011`), so an AGA machine displays the OCS layout correctly. The starter's list shows the minimum shape.

Nothing included before `engine_main.asm` may emit bytes. AmigaDOS starts the program at the first byte of the first code hunk, so a table in a constants file would become the entry point. `tests/test_engine_contract.py` checks this.

6. Hooks: Where the Engine Calls the Game

At fixed points in the frame, level and undo cycles, the engine calls a `Game*Hook`. Each one is a routine in your `game_hooks.asm`, or a binding to an existing routine (`GameLogicHook = PlayerLogic`). The starter defines them all as `rts`. Unless the contract says otherwise, a hook may change `d0-d7/a0-a4` and must leave `a5/a6` alone.

Group	Hooks	Called
Boot and level	`GameInitHook`, `GameSetupHook`, `GameLevelInitHook`, `GameShowPlayerHook`, `GameLevelDoneHook`	boot, game start, level load, level reveal, every frame (level complete test)
Frame drawing	`GameDrawPriorityHook`, `GameRestoreOverlapsHook`, `GameDrawHook`	the draw pass, in the order of the Blit Order Guide
Frame logic	`GameLogicHook`, `GamePostLogicHook`, `GameActorPhaseHook`, `GameDrawEntitiesHook`	after the draw pass, in that order

Group	Hooks	Called
Undo	`GameUndoAllowedHook`, `GameSnapshotSaveHook`, `GameUndoEraseHook`, `GameSnapshotRestoreHook`, `GameUndoRedrawHook`	F9 rewind
Entities	`GameEnemySheetHook`, `GameEnemyOverlayHook`	every enemy blit
Actors	`GamePushMoveHook`, `GameActorImpactHook`	crate push and landing
Misc	`GameUnpauseHook`, `GameDebugCountHook`	pause, debug overlay

Anything that draws must follow the Blit Order Guide. The drawing hooks run at fixed points in a single-buffered frame, and the order is what keeps them from tearing.

7. Adding Features: Include the Subsystem

Engine features are not switched on with flags. You include the file that implements one, satisfy what it needs, and call it (or let the frame call it).

Feature	Include	You also provide
Tile map, scrolling, camera	`tmx_renderer.asm`, `dirty_tiles.asm`, `copper_sky.asm`	`DisplayScreen/NonDisplayScreen` (`LEVEL_SCREEN_SIZE` each, Chip RAM), level data (§8)
Player BOB	`bob_compositor.asm`, `bob_mirror.asm`	the `Player_*` struct (copy Duck Rescue's `struct.asm`), the player sheets
Pushable, falling crates	`actors.asm`, `actor_physics.asm`	`GamePushMoveHook`, `GameActorImpactHook`, `AudioPlayCratePush/Impact`
Enemies, friends, pickups	`entities.asm`	`entity_struct.i`, `entity_variables.i`, the three `MAX_ACTIVE_*` sizes, the sprite sheets, both enemy hooks
Rising water, air bubble	`water_effects.asm`, `bubble_sprite.asm`	a water layer in the level; for the bubble, `PLAYER_HEAD_Y_OFFSET`, `COLOR29–31` and a per-frame call to `UpdateAirBubble`
Undo and VHS rewind	`undo.asm`, `vhs_rewind.asm`	a `Snap_*` struct that opens with `engine_snapshot.i`, the undo hooks
Screens and state machine	`gamestatus.asm`, `engine/screens/*.asm`	the data bindings (title art, palettes, instruction text)

The fastest way to find what a subsystem needs is to let the build tell you. Include the file and run:

```
python tools/build.py --game my_game --explain
```

On a failed build, `--explain` lists every missing symbol, the engine file that needs it, and where Duck Rescue defines it. Copy or bind each one, then build again. The starter itself was built this way.

Include order matters: a file can use only the macros and equates defined above it. Follow the order in Duck Rescue's `main.asm`: headers and constants (`amiga_hw.i` first), then `engine_main.asm`, then system, graphics, gameplay, game code and screens, then audio and data.

Scrolling sideways

The engine's tilemap renderer (`tmx_renderer.asm`) scrolls vertically, one screen wide. For a level wider than the screen, include `engine/graphics/hscroll.asm` and add `include "hscroll_variables.i"` to your `variables.asm` after `engine_variables.i`. Before it, define `HS_MAP_ROWS`, `HS_MAP_COLS`, `HS_SHEET_COLS` and `HS_SHEET_ROW_BYTES`; provide `HsTileSheet`, `HsTileMask`, two `HS_BUF_SIZE` Chip BSS buffers (`HsDisplayBuf`,

HsCleanBuf), and HsCopperScroll/HsCopperPlanes entries after a Copper WAIT just above the display window. Point HsMapBack, HsMapMid and HsMapFront at your word maps and call HsInit, then HsSetCamera early in every frame. games/meadow_run/ shows all of it.

8. Levels and Assets

Levels are drawn in Tiled and exported by export_level.py into tile sheets, layer maps and an assembly LevelDef per level. Sprites, BOBs, palettes, sound and tables are converted by the scripts under games/<game>/tools/. The Asset Pipeline Guide covers every format and converter.

For a new game:

1. Copy games/duck_rescue/tools/ and game_palette.json into your game. tools/build.py runs the converters listed in its ASSET_PIPELINE from **your** game's tools/ folder, by those file names, and skips any that are missing.
2. Edit game_palette.json for your palette. No Python changes are needed for a new palette.
3. Adapt the converters whose sheet sizes or layouts differ. The bitplane packing (amiga_planar.py) is shared and needs no changes.
4. Bind the level data the engine falls back to: LevelDefaultPalette, LevelDefaultPlatformMap, LevelDefaultEnemyList, LevelDefaultMusic.

9. Keeping It Working

Check	Command	Catches
Contract tests	python -m unittest discover tests	missing hooks, game names in engine code, code that falls off the end of a file, a non-code entry point
Converter goldens	the same command	a converter or source asset that changed by accident; accept intended changes with python tests/update_golden.py
Executable goldens	the same command	any change to the generated code; accept with python tests/update_golden.py --executables *after* testing in the emulator
Build with tests first	python tools/build.py --game my_game --check	builds only if the tests pass

To pin your game's converter outputs, run python tests/update_golden.py --game my_game once it builds. To pin its executable, add it to GAMES in tests/test_build_golden.py and run python tests/update_golden.py --executables.

10. A Path From Starter to Game

1. **Boot and quit:** the starter, renamed (§2).
2. **A screen:** the starter already shows one (5 bitplanes, a Copper gradient, text); change its depth, palette and contents to your game's, with the palette from your game_palette.json.
3. **A level:** include the tile map subsystem, export one Tiled map, and bind the LevelDefault* data.
4. **A player:** the Player_* struct, the BOB compositor, and your movement in GameLogicHook.
5. **Things to meet:** entities or actors, each with its hooks.
6. **A structure:** gamestatus.asm for title, levels and end screens, or your own GameStatusRun.
7. **Polish:** undo, water, audio.

Build and run after each step, on the A500 profile first. It is the slowest machine and the one most likely to show a timing problem.